

Key-Value Database

System Design — Deep Dive Notes

Consistent Hashing · Replication · Vector Clocks
Gossip Protocol · Merkle Tree · Eventual Consistency

TOPICS COVERED

1. Goals & Architecture

2. Consistent Hashing & Partitioning

3. Virtual Nodes

4. Replication & Load Balancers

5. Vector Clocks — Data Versioning

6. Gossip Protocol — Deep Dive

7. Merkle Tree — Sync Checking

8. Eventual Consistency & DynamoDB

Table of Contents

1. Goals & Core Architecture	3
2. Consistent Hashing & Partitioning	3
3. Virtual Nodes	4
4. Replication & Preference List	4
5. Load Balancer Types	5
6. Get & Put Operations — Quorum	5
7. Data Versioning — Vector Clocks (Deep Dive)	6
8. Gossip Protocol (Deep Dive)	7
9. Merkle Tree (Deep Dive)	7
10. Eventual Consistency & How It Works	8
11. DynamoDB Architecture Overview	9
12. Quick Revision & Interview Questions	9

1. Goals & Core Architecture

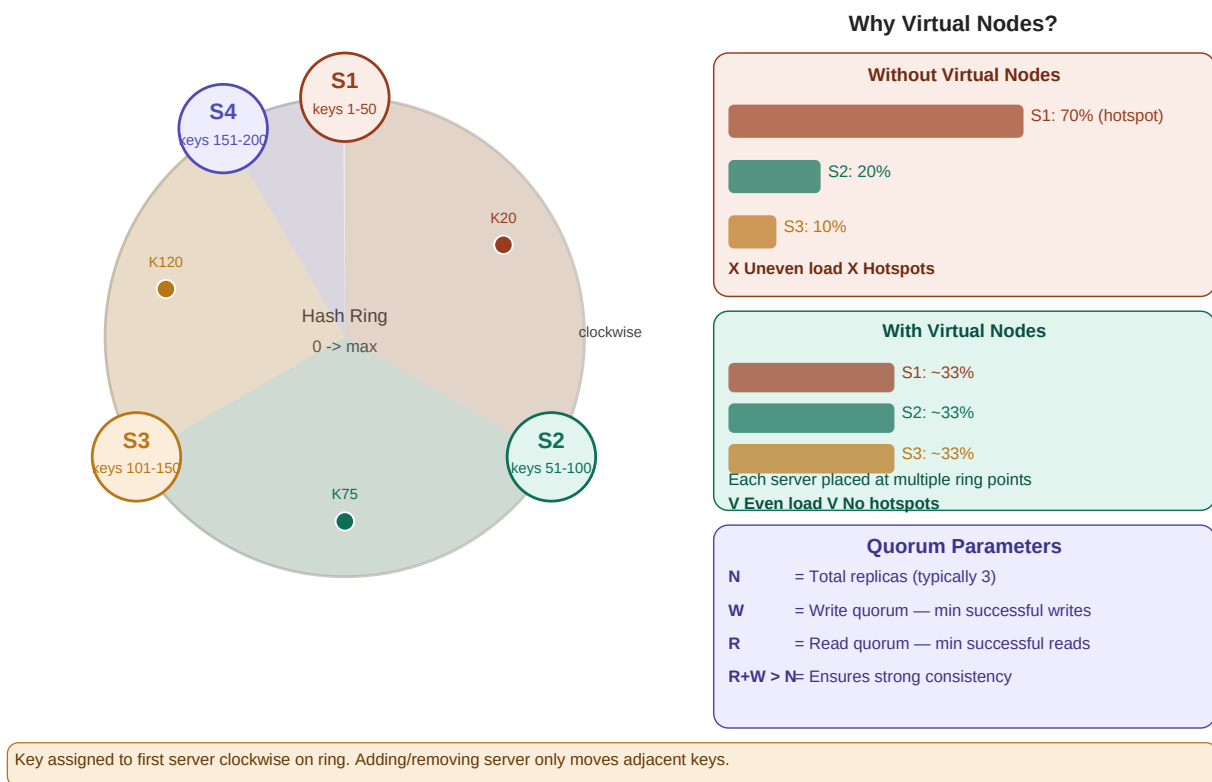
A **key-value database** is a distributed hash table at planet scale. Amazon's DynamoDB, Apache Cassandra, and Riak are real-world implementations. The design must solve three goals simultaneously, and every component we build traces back to one of these goals:

Goal	What it means	How achieved
Scalability	Handle billions of users, petabytes	Consistent hashing + partitioning
Decentralization	No single point of failure	Replication + gossip + preference list
Eventual Consistency	Data converges, not always instant	Vector clocks + quorum (R,W,N)

2 & 3. Consistent Hashing, Partitioning & Virtual Nodes

Consistent hashing places both servers and data keys on a circular ring (the hash space). Each key is assigned to the first server clockwise. This solves two problems: it distributes data across servers, and adding/removing a server only moves $\sim 1/N$ of all keys — not everything.

Consistent Hashing — Partitioning Virtual Nodes



Hash ring with 4 servers · Coloured arcs show key ranges · Bar charts compare load with/without virtual nodes

Why Virtual Nodes are Essential

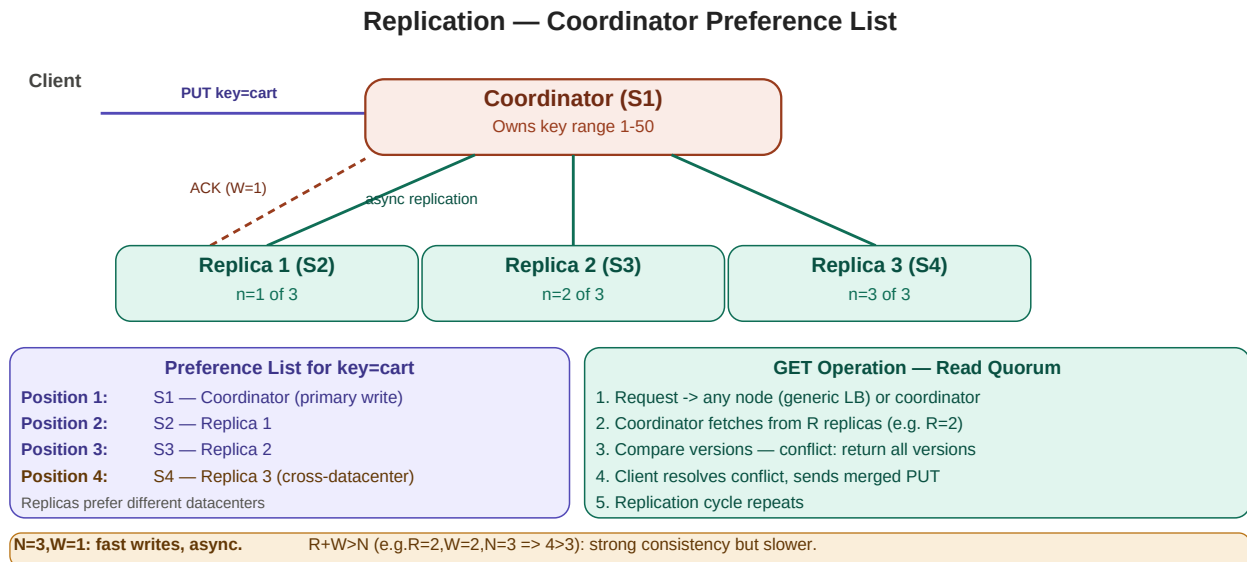
If you place each server at one point on the ring, the distribution depends entirely on where `hash(server_id)` lands. By chance, S1 might own 70% of the ring. **Virtual nodes** solve this by giving each physical server multiple positions on the ring.

- S1 gets positions at 0, 120, 240 degrees — 3 evenly spread arcs of the ring

- S2 gets positions at 40, 160, 280 degrees — interleaved with S1's positions
- Each physical server ends up owning approximately $1/N$ of all keys
- Cassandra default: 256 virtual nodes per server for ultra-smooth distribution
- When a server is added, it takes a small slice from each existing server's vnodes

4. Replication & Preference List

One copy of data is a single point of failure. **Replication factor $N=3$** means every key has 3 copies: one on the coordinator and two on replica servers. The **preference list** is a per-key ordered list recording exactly which servers hold each copy.



PUT: coordinator stores + fans out async replicas · ACK after W confirms · GET: read from R replicas · preference list on left

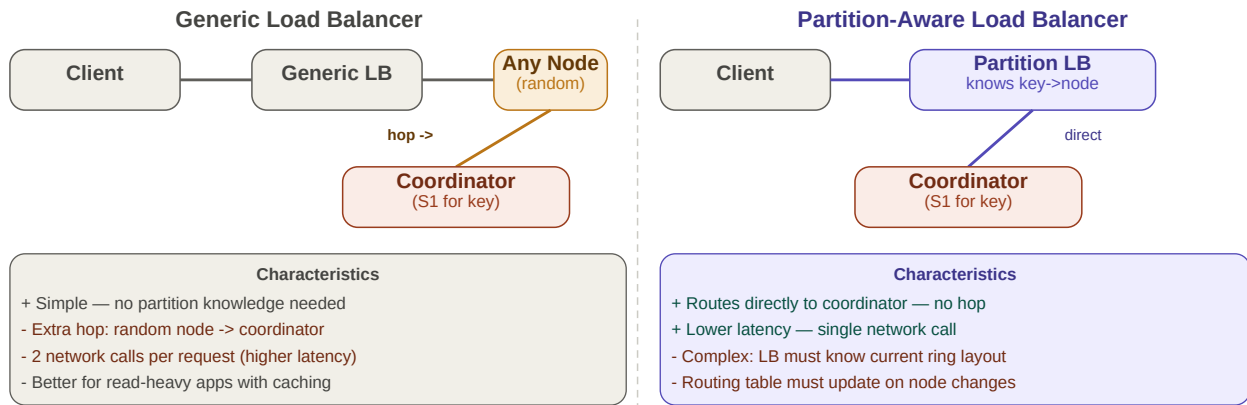
Preference List Details

- Position 1 (coordinator): handles all writes for this key range
- Positions 2–N: replicas — receive async copies after coordinator writes
- Cross-datacenter: preferably replicas in different datacenters for disaster resilience
- On coordinator failure: position 2 becomes the new coordinator automatically
- Preference list is maintained and gossiped to all nodes in the cluster

5. Load Balancer Types — Generic vs Partition-Aware

Requests arrive at the cluster through a load balancer. The type of load balancer determines whether the request lands directly on the coordinator or requires an extra network hop.

Load Balancer Types — Generic vs Partition-Aware



Generic LB: request may land on wrong node, extra hop needed · Partition-aware LB: routes directly to coordinator

Aspect	Generic Load Balancer	Partition-Aware Load Balancer
Knowledge	No partition awareness	Knows hash ring layout
Routing	Sends to any node	Sends directly to key's coordinator
Latency	Higher (2 hops)	Lower (1 hop)
Complexity	Simple	Complex — must track ring changes
Used by	Simple setups, REST APIs	DynamoDB, Cassandra clients
Failure handling	Any node can forward	Must recompute on topology change

6. Get & Put Operations — Quorum (R, W, N)

The parameters **N** (total replicas), **W** (write quorum), and **R** (read quorum) let you tune the trade-off between consistency and availability. The formula $R + W > N$ guarantees that read and write sets overlap — ensuring strong consistency.

Configuration	N=3,W=1,R=1	N=3,W=2,R=2	N=3,W=3,R=1
R+W	2 — no overlap	4 > 3 — overlap	4 > 3 — overlap
Consistency	Eventual (weak)	Strong	Strong
Write speed	Fast (async)	Medium (sync 2)	Slow (sync all)
Availability	Highest	Medium	Lowest
Best for	Social feeds, DNS	Banking, cart	Critical config

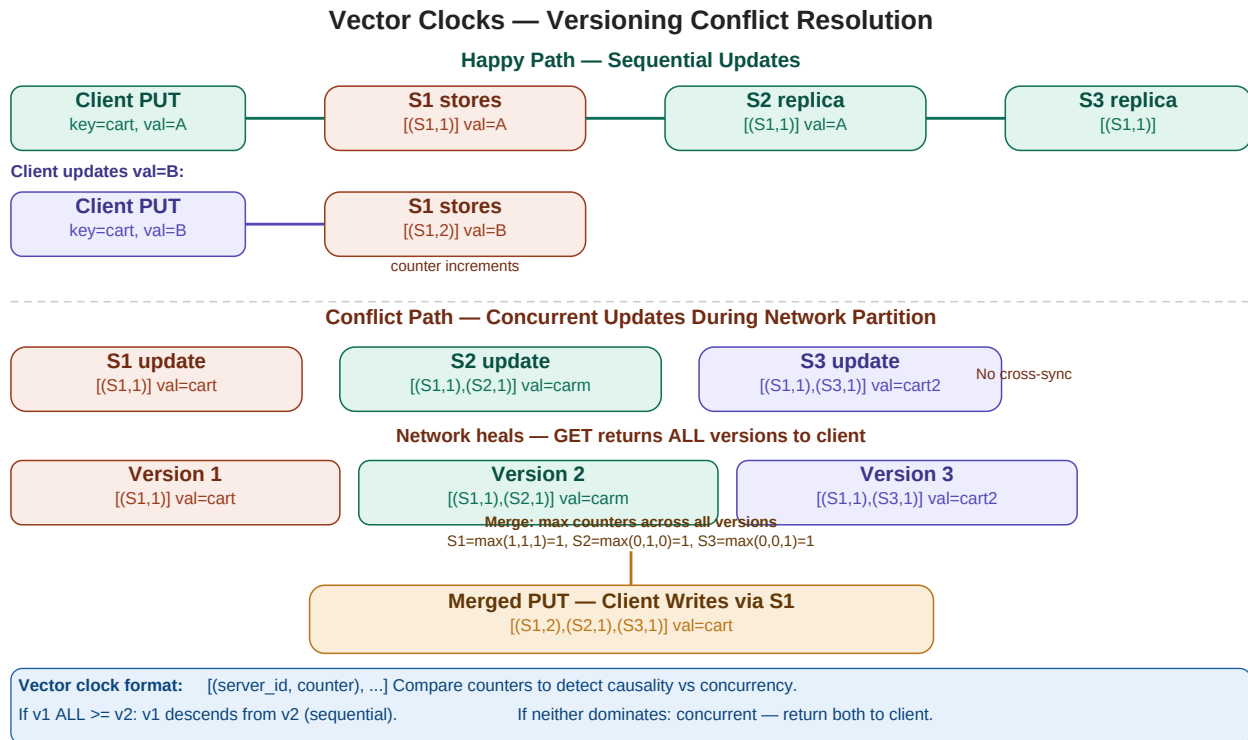
PUT operation: client sends to coordinator. Coordinator writes locally, fans out to N-1 replicas async. Returns success after W confirms.

GET operation: coordinator collects responses from R replicas, compares versions. If consistent: return value. If conflict: return all versions to client.

Sloppy quorum: if a coordinator is unavailable, a "hinted handoff" node temporarily handles writes and passes them on recovery.

7. Data Versioning — Vector Clocks (Deep Dive)

When replicas accept writes concurrently (during a network partition or coordinator failure), they diverge. **Vector clocks** are the mechanism to track version history, detect causality, and identify conflicts that require client resolution.



Happy path: sequential writes chain cleanly · Conflict path: concurrent writes return all versions · client resolves and sends merged PUT

How Vector Clocks Work — Step by Step

Structure

A vector clock is a list of (**server_id**, **counter**) tuples. Every server that writes to a key appends its ID and increments its counter. Example: [(S1,2),(S2,1)] means S1 wrote twice, S2 wrote once.

Causality Detection

- Version A causes Version B if every counter in A is <= the corresponding counter in B
- Example: [(S1,1)] causes [(S1,2)] — S1 just incremented. Sequential update, no conflict.
- Example: [(S1,1)] and [(S1,1),(S2,1)] — neither dominates — concurrent writes, conflict!
- Concurrent = same base state, two servers wrote independently without seeing each other

Conflict Resolution Process

- GET request returns ALL conflicting versions to the client application
- Client applies business logic: last-write-wins (use highest timestamp), or merge (Amazon cart: union of all items)
- Client submits the resolved value as a new PUT with the merged vector clock
- The merged PUT then propagates normally through the replication pipeline
- All replicas converge to the merged value after replication completes

Amazon cart example: User adds item A on one device, item B on another (during partition). Vector clocks detect two concurrent writes. Resolution: merge = keep both A and B in cart.

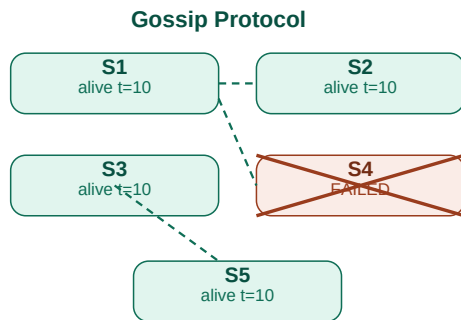
Last-Write-Wins (LWW): simpler but loses data — the later timestamp overwrites earlier. Fine for DNS, caches. Bad for shopping carts.

Vector clock truncation: clocks can grow large over time. Systems truncate old entries by (server, timestamp) — slight risk of false concurrency detection.

8. Gossip Protocol — Deep Dive

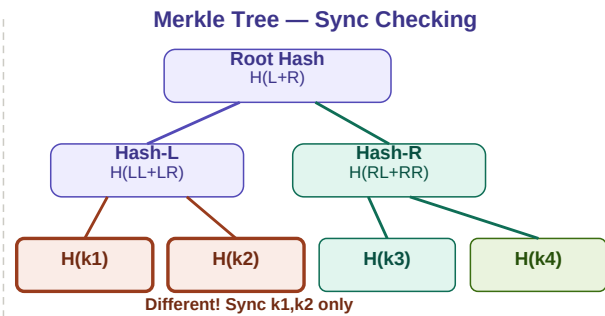
In a decentralized system, there is no central server tracking which nodes are alive. **Gossip protocol** solves this by having every node periodically share its knowledge of cluster state with random peers. Information spreads like a rumour — exponentially fast.

Gossip Protocol Merkle Tree



How it works:

1. Each node periodically sends heartbeat to random peers
2. Heartbeat: timestamp counter, key ranges, node status
3. Info spreads in $O(\log N)$ rounds — gossip-like
4. No heartbeat increment = node detected as failed
5. Traffic rerouted via preference list



How it works:

1. Compare root hashes first
2. Same root = fully in sync
3. Different = traverse tree down
4. Find which leaf keys differ
5. Sync **ONLY** those keys

Gossip: $O(\log N)$ propagation — all nodes learn cluster state in $\log N$ rounds without central coordinator.

Merkle Tree: $O(\log N)$ sync check — binary search to find differing keys instead of scanning every key.

Gossip: dashed heartbeat arrows between nodes · S4 failed (crossed) detected by missing heartbeat · Merkle tree on right

Gossip Mechanics — Step by Step

What each server tracks

- Membership list: table of all known servers with their last-seen heartbeat counter
- For each server: ID, IP address, key ranges owned, health status, version
- This list is the single source of truth for cluster topology

How gossip spreads

- Every T seconds (typically 1s), each server picks 1-2 random peers and sends its membership list
- Receiver merges the received list with its own — taking the more recent heartbeat for each entry
- After k rounds, every node has seen the information — $k \sim \log_2(N)$ for N nodes
- For 1000 nodes: ~ 10 rounds = ~ 10 seconds for full propagation

Failure Detection

- Each server's heartbeat counter increments every second
- If a server's counter has not incremented for T_{fail} seconds — it is suspected dead
- Suspected dead: traffic rerouted to next entry in preference list via hinted handoff
- Confirmed dead after T_{confirm} : node removed from ring, keys redistributed

9. Merkle Tree — Efficient Replica Sync (Deep Dive)

After a network partition heals, replicas may have diverged. Naively, you would compare every key across all replicas — for millions of keys this is prohibitively expensive. **Merkle trees** solve this by compressing the data into a binary hash tree that enables $O(\log N)$ sync detection.

How Merkle Trees Work — Step by Step

Building the tree

- Leaf nodes: $\text{hash}(\text{key_value_pair})$ for each key in the replica's range
- Parent node: $\text{hash}(\text{left_child_hash} + \text{right_child_hash})$
- Continue up until root = hash of all data in the range
- If ANY leaf changes, every ancestor hash changes too — root will be different

Sync check between coordinator and replica

- Step 1: Compare root hashes. If equal — replica is fully in sync, done
- Step 2: If root differs, compare left and right children of root
- Step 3: Recurse into the subtree that differs (binary search)
- Step 4: Reach leaf nodes — identify the exact keys that differ
- Step 5: Send only those differing keys over the network
- Complexity: $O(\log N)$ comparisons to find differences in N keys

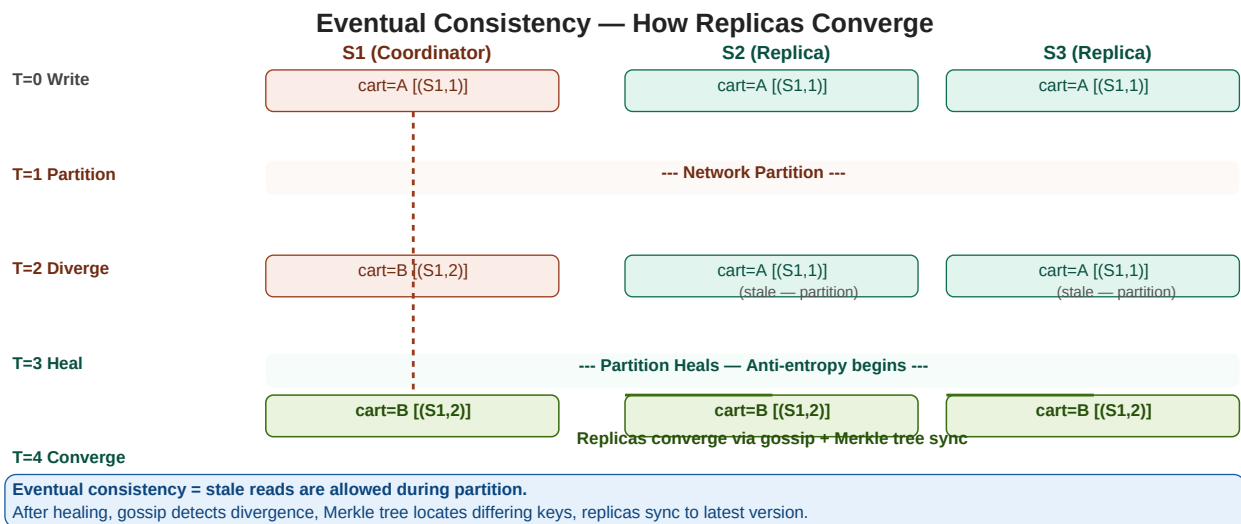
Example: 1 million keys. Without Merkle: compare 1M key-value pairs. With Merkle: compare ~20 hash values to find the exact few keys that differ.

Cassandra uses Merkle trees for anti-entropy repair — periodically checks replicas and syncs differences.

Each partition of the ring has its own Merkle tree. Trees are rebuilt periodically (configurable).

10. Eventual Consistency — How It Actually Works

Eventual consistency means replicas may temporarily diverge but will converge to the same value after sufficient time and operations. This is the trade-off for staying available during network partitions (AP from CAP theorem).



Timeline: all start in sync -> partition -> replicas diverge -> partition heals -> gossip + Merkle sync -> all converge

The Anti-Entropy Pipeline

Three mechanisms work together to drive replicas toward consistency:

- 1. **Gossip protocol** detects that S2 and S3 are behind — their heartbeats include their Merkle tree root hash
- 2. **Merkle tree comparison** identifies that the cart key differs between S1 and S2/S3
- 3. **Read repair:** when a GET fetches from R replicas, coordinator detects stale replica and updates it
- 4. **Hinted handoff:** writes accepted during failure stored as "hints" and replayed on recovery
- 5. **Scheduled repair:** background jobs periodically run full Merkle-based anti-entropy across all replicas

Stale read window: a client can read stale data for as long as the partition lasts (could be seconds or minutes).

Bounded staleness: some systems offer "read your own writes" guarantee by routing reads to the coordinator.

DynamoDB allows configuring "Strongly Consistent Reads" (reads from coordinator only) at the cost of higher latency.

11. DynamoDB Architecture Overview

Amazon DynamoDB is the production implementation of these concepts, originally described in the **Dynamo paper (2007)**. Every design choice traces directly to the concepts in this document.

DynamoDB Component	Concept Used	Details
Partition key hashing	Consistent hashing	MD5 hash maps key to partition on ring
Partition replication	N=3 replication	3 replicas across availability zones

DynamoDB Component	Concept Used	Details
Coordinator routing	Partition-aware LB	Client SDK routes to correct coordinator
Write acknowledgment	Quorum W	Default W=2 for write (majority of 3)
Read consistency	Quorum R	Eventually consistent R=1, strong R=2
Conflict detection	Vector clocks	Internal version tracking per item
Node failure detection	Gossip protocol	All nodes gossip health every second
Replica sync	Merkle trees	Anti-entropy repair runs periodically
CAP choice	AP	Stays available during partitions, eventual consistency

12. Quick Revision & Interview Questions

Quick Revision

- > KV database = distributed hash table. Goals: scalability, decentralization, eventual consistency.
- > Consistent hashing: ring-based. Key -> first server clockwise. Adding server moves $\sim 1/N$ keys only.
- > Virtual nodes: each server gets multiple ring positions for even load. Cassandra: 256 vnodes/server.
- > Replication $N=3$: coordinator holds primary, $N-1$ replicas get async copies.
- > Preference list: ordered list of nodes for each key range. On failure, use next in list.
- > Generic LB: sends to any node, needs a hop to coordinator. Higher latency.
- > Partition-aware LB: routes directly to coordinator. Lower latency, more complex.
- > $W=1, R=1$: eventually consistent (fast). $R+W>N$ (e.g. $R=2, W=2, N=3$): strong consistency.
- > Vector clock = $[(server_id, counter)]$. Each server increments its own counter on write.
- > If $v1$ counters ALL $\geq v2$: $v1$ descends from $v2$. Sequential, no conflict.
- > If neither dominates: concurrent writes — return both versions to client to resolve.
- > Client resolves conflict (LWW or merge) and sends a new PUT with merged vector clock.
- > Gossip: each node sends heartbeat to random peers every 1s. Spreads in $O(\log N)$ rounds.
- > Missing heartbeat counter = suspected failed. Traffic rerouted via hinted handoff.
- > Merkle tree: hash tree over keys. Compare root first -> traverse to find differing leaf keys.
- > Eventual consistency: replicas diverge during partition, converge via gossip + Merkle repair.
- > DynamoDB = AP system. Stays available during partitions. Data eventually consistent.

Interview Questions

- 1. Explain consistent hashing. Why is it better than modulo hashing for distributed systems?
- 2. What are virtual nodes and why are they needed?
- 3. Walk me through a PUT operation in a key-value store from client to all replicas.
- 4. What is the preference list and how does failover work when coordinator fails?
- 5. What is the difference between a generic and partition-aware load balancer?
- 6. Explain quorum parameters N, W, R . When would you use $R=2, W=2, N=3$?
- 7. What is a vector clock? How does it detect concurrent vs sequential writes?
- 8. Walk me through a conflict scenario: two servers write the same key simultaneously.

- 9. How does last-write-wins differ from merge-based conflict resolution?
- 10. Explain how the gossip protocol detects node failure without a central coordinator.
- 11. What does a Merkle tree store and how does it speed up replica sync?
- 12. What is eventual consistency? How does a system recover after a network partition?
- 13. What is read repair and hinted handoff?
- 14. Design DynamoDB's add-to-cart feature. Which CAP trade-off do you choose and why?

Key Terms

Consistent Hashing	Ring-based partitioning — key goes to first server clockwise
Virtual Node (vnode)	Multiple ring positions per server for even key distribution
Coordinator	Server responsible for a key range — handles writes, manages replicas
Preference List	Ordered list of nodes for a key range (coordinator first, replicas after)
N, W, R	N=replicas, W=write quorum, R=read quorum. $R+W>N$ = strong consistency
Sloppy Quorum	Accept writes on healthy node even if coordinator is down (hinted handoff)
Hinted Handoff	Store write on substitute node as "hint"; replay to failed node on recovery
Vector Clock	[(server_id, counter)] list tracking which server wrote, how many times
Causality	If v1 counters $\geq$ v2 for all servers: v1 descends from v2 (sequential)
Concurrent Writes	Neither clock dominates the other — happened independently, conflict
Last-Write-Wins	Conflict resolution: keep highest timestamp. Simple but loses data
Gossip Protocol	Peer-to-peer heartbeat propagation. $O(\log N)$ rounds for full cluster
Heartbeat Counter	Incrementing counter per node. Missing increment = suspected failure
Anti-entropy	Background process to find and fix inconsistencies between replicas
Merkle Tree	Binary hash tree over keys. $O(\log N)$ sync check — compare root first
Read Repair	On GET: coordinator detects stale replica and updates it inline
Eventual Consistency	Replicas may diverge temporarily but converge given time and operations
AP System	Chooses Availability + Partition Tolerance. Consistent given enough time.